

Documentación API Principal – Proyecto Final Supermercado

Integrante Responsable

Gustavo Sandoval

Descripción General

Esta API representa el núcleo principal del sistema del supermercado.

Su función es manejar toda la lógica del negocio:

- Productos
- Clientes
- Proveedores
- Compras
- Ventas
- Inventario
- Estados de ventas
- Reportes

La API fue desarrollada utilizando:

- C#
 - ASP.NET Core Web API
 - Entity Framework Core
 - SQL Server
 - Swagger
-

Arquitectura General

El flujo principal del sistema es:

```

Cliente/Swagger
    ↓
Controllers
    ↓
Entity Framework Core
    ↓
SQL Server

```

Tecnologías Utilizadas

Tecnología	Uso
ASP.NET Core	API REST
Entity Framework Core	ORM y acceso a datos
SQL Server	Base de datos
Swagger	Documentación y pruebas
LINQ	Consultas y reportes

Estructura del Proyecto

```
Controllers/  
Models/  
Data/  
DTOs/  
Migrations/
```

Base de Datos

Base utilizada

```
SupermercadoDB
```

Tablas Principales

Productos

Contiene:

- Id
- Nombre
- Descripción
- Precio
- Stock

Cientes

Contiene:

- Id
 - Nombre
 - Email
 - Telefono
-

Proveedores

Contiene:

- Id
 - Nombre
 - Contacto
 - Telefono
-

Compras

Representa compras realizadas a proveedores.

Incluye:

- Fecha
 - Total
 - Relación con proveedor
 - Detalles de compra
-

Ventas

Representa ventas realizadas a clientes.

Incluye:

- Fecha
 - Total
 - Estado
 - Relación con cliente
 - Detalles de venta
-

DetallesCompra

Guarda:

- Producto
 - Cantidad
 - Precio unitario
 - Subtotal
-

DetallesVenta

Guarda:

- Producto
 - Cantidad
 - Precio unitario
 - Subtotal
-

Funcionalidades Implementadas

1. CRUD Productos

Endpoints:

```
GET /api/productos
GET /api/productos/{id}
POST /api/productos
PUT /api/productos/{id}
DELETE /api/productos/{id}
```

Permite:

- Crear productos
 - Editar productos
 - Eliminar productos
 - Consultar productos
-

2. CRUD Clientes

Endpoints:

```
GET /api/clientes
GET /api/clientes/{id}
POST /api/clientes
PUT /api/clientes/{id}
DELETE /api/clientes/{id}
```

3. CRUD Proveedores

Endpoints:

```
GET /api/proveedores
GET /api/proveedores/{id}
POST /api/proveedores
PUT /api/proveedores/{id}
DELETE /api/proveedores/{id}
```

4. Registro de Compras

Endpoint:

```
POST /api/compras
```

Funcionalidad:

- Registra compras a proveedores
- Actualiza inventario automáticamente
- Aumenta stock de productos
- Calcula subtotales y total

Flujo:

```
Proveedor
↓
Compra
↓
DetalleCompra
↓
Stock aumenta
```

5. Registro de Ventas

Endpoint:

```
POST /api/ventas
```

Funcionalidad:

- Registra ventas a clientes
- Verifica stock disponible
- Descuenta inventario automáticamente
- Calcula subtotales y total

Flujo:

```
Cliente
  ↓
Venta
  ↓
DetalleVenta
  ↓
Stock disminuye
```

6. Estados de Venta

Endpoint:

```
PUT /api/ventas/{id}/estado
```

Estados posibles:

```
Pendiente
Procesando
Enviado
Entregado
Cancelado
```

Objetivo:

Permitir integración futura con Azure Queue Storage y envío de notificaciones.

Ejemplo:

Venta → Estado cambiado → Azure Queue → Correo

7. Reportes

Ventas por Fecha

Endpoint:

GET /api/reportes/ventas-por-fecha

Retorna:

- Cantidad de ventas
 - Total vendido
 - Listado de ventas
-

Ventas por Cliente

Endpoint:

GET /api/reportes/ventas-por-cliente/{clienteId}

Retorna:

- Total comprado por cliente
 - Historial de ventas
-

Compras por Proveedor

Endpoint:

GET /api/reportes/compras-por-proveedor/{proveedorId}

Retorna:

- Total comprado a proveedor
 - Historial de compras
-

Productos Más Vendidos

Endpoint:

```
GET /api/reportes/productos-mas-vendidos
```

Retorna:

- Nombre del producto
 - Cantidad vendida
-

Inventario Automático

La lógica de inventario funciona automáticamente.

Compras

```
Compra → stock aumenta
```

Ventas

```
Venta → stock disminuye
```

Swagger

Swagger se utiliza para:

- Probar endpoints
- Visualizar documentación
- Simular operaciones
- Validar respuestas

URL local:

```
https://localhost:7239/swagger
```

Flujo Recomendado de Uso

Paso 1

Crear proveedor

Paso 2

Crear producto

Paso 3

Crear cliente

Paso 4

Registrar compra

Paso 5

Verificar stock

Paso 6

Registrar venta

Paso 7

Cambiar estado de venta

Paso 8

Consultar reportes

Integración Esperada con Otros Módulos

API E-Commerce

Consumirá:

- Productos
- Ventas
- Clientes
- Catálogo

Azure Queue Storage

Se integrará principalmente con:

```
PUT /api/ventas/{id}/estado
```

Cuando el estado cambie:

```
Enviado  
Entregado
```

podrán generarse mensajes hacia Azure Queue.

Docker e Infraestructura

La API ya está preparada para:

- Dockerizar
 - Conectarse a SQL Server
 - Ejecutarse mediante contenedores
-

Problemas Solucionados Durante el Desarrollo

Error de ciclos JSON

Se solucionó utilizando:

```
ReferenceHandler.IgnoreCycles
```

Error de Entity Framework

Se solucionó instalando correctamente:

- Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - Microsoft.EntityFrameworkCore.Design
-

Migraciones

Se utilizaron:

Add-Migration
Update-Database

Estado Actual del Proyecto

Completado

- CRUD Productos
- CRUD Clientes
- CRUD Proveedores
- Compras
- Ventas
- Inventario
- Estados
- Reportes
- Swagger
- SQL Server
- Relaciones Entity Framework

Mejoras Futuras Recomendadas

- JWT Authentication
- Roles y permisos
- DTOs completos
- Validaciones avanzadas
- Paginación
- Logs
- Azure Queue real
- Docker completo
- Integración frontend

Conclusión

La API principal quedó preparada como núcleo del sistema del supermercado.

Actualmente permite:

- Manejar inventario

- Procesar ventas
- Registrar compras
- Generar reportes
- Gestionar clientes y proveedores
- Integrarse con otros servicios

La estructura fue diseñada para ser escalable y fácil de integrar con los demás módulos del proyecto final.